Sesión 3

Representación del Texto: BOW, TF-IDF y N-gramas

Carlos César Sánchez Coronel

NLP

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

¿Por qué representar el texto numéricamente?

- Los modelos de machine learning operan con números, no con texto directamente.
- Cada documento debe convertirse en un **vector numérico**.
- La representación elegida afecta:
  - Capacidad semántica.
  - Dimensionalidad.
  - Rendimiento de los modelos.
- En esta sesión: representaciones **dispersas** clásicas.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Definición e intuición

Idea

Representar cada documento como una **bolsa** de sus palabras, ignorando el orden pero manteniendo la frecuencia.

- El contenido se infiere por las palabras que contiene.
- **Vocabulario**: conjunto de todas las palabras únicas del corpus.

Construcción paso a paso

1. Crear vocabulario con todas las palabras únicas del corpus.
2. Asignar un índice a cada palabra.
3. Construir la matriz documento-término: cada fila = documento, cada columna = palabra, valor = frecuencia.

Ejemplo (3 documentos)

d1: .el gato persigue al ratón"
d2: .el perro persigue al gato"
d3: .el ratón huye del gato"

Ejemplo numérico de BOW

Vocabulario: al, el, gato, huye, del, perro, persigue, ratón
Matriz de frecuencias:

$$\mathbf{X} = \begin{pmatrix} d_1: & [1,1,1,0,0,0,1,1] \\ d_2: & [1,1,1,0,0,1,1,0] \\ d_3: & [0,1,1,1,1,0,0,1] \end{pmatrix}$$

Limitaciones de BOW

- **Pérdida de orden**: .el gato persigue al ratón"vs .el ratón persigue al gato"tienen el mismo vector.
- **Alta dimensionalidad**: vocabulario enorme, vectores muy dispersos.
- **Falta de semántica**: sinónimos (gato/felino) son independientes.
- **Palabras comunes dominan**: .el", "de.aparecen en todos.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Definición de n-gramas

N-grama

Secuencia contigua de n ítems (palabras o caracteres) extraída de un texto.

- **Unigramas** ($n = 1$): palabras individuales (BOW).
- **Bigramas** ($n = 2$): pares de palabras consecutivas.
- **Trigramas** ($n = 3$): tripletes.
- También n-gramas de caracteres.

Ejemplo con bigramas

Frase: .el gato persigue al ratón"

- Bigramas: .el gato", "gato persigue", "persigue al", .al ratón".
- Capturan contexto local que los unigramas ignoran.
- Útil para detectar frases hechas o patrones.

Compensación: orden vs dimensionalidad

- Los n-gramas aumentan drásticamente la dimensionalidad.
- Para un vocabulario de m palabras, hay hasta m^n n-gramas posibles.
- En la práctica, muchos no aparecen y se pueden filtrar.
- Compromiso: capturan algo de orden sin modelar estructura completa.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Definición e intuición de TF-IDF

TF-IDF (Term Frequency - Inverse Document Frequency)

Ponderación que da mayor peso a términos frecuentes en un documento pero raros en el corpus.

- Penaliza palabras comunes (artículos, preposiciones).
- Resalta términos distintivos del documento.

Componentes de TF-IDF

TF (Term Frequency)

- Frecuencia bruta: $tf(t,d) = f_{t,d}$
- Logarítmica: $tf(t,d) = \log(1 + f_{t,d})$
- Normalizada: $tf(t,d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$

IDF (Inverse Document Frequency)

$$idf(t) = \log\left(\frac{N}{df(t)}\right)$$

- N: número total de documentos.
- $df(t)$: número de documentos que contienen t.
- Suavizado: $\log\left(\frac{N+1}{df(t)+1}\right) + 1$

Peso TF-IDF: $tfidf(t,d) = tf(t,d) \times idf(t)$

Ejemplo numérico de TF-IDF

Retomando el corpus anterior:

- $N = 3$, término "gato": $df = 3$, $idf = \log(3/3) = 0$.
- Término "perro": $df = 1$, $idf = \log(3/1) \approx 1{,}099$.
- Término "persigue": aparece en d1 y d2, $df = 2$, $idf \approx 0{,}405$.

Matriz TF-IDF (TF bruta):

$$\begin{pmatrix} d_1: & [0, 0, 0, 0, 0, 0, 0{,}405, 0{,}405] \\ d_2: & [0, 0, 0, 0, 0, 1{,}099, 0{,}405, 0] \\ d_3: & [0, 0, 0, 1{,}099, 1{,}099, 0, 0, 0{,}405] \end{pmatrix}$$

Normalización de vectores TF-IDF

- Es común normalizar cada vector documento a norma unitaria (por ejemplo, L2).
- La similitud del coseno se convierte en producto punto.

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

- Así, la longitud del documento no influye en la similitud.

Interpretación geométrica

- Cada documento es un punto en un espacio de alta dimensión.
- La similitud entre documentos se mide con el **coseno del ángulo**:

$$\text{sim}(d_i, d_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \|\mathbf{v}_j\|}$$

- Si los vectores están normalizados, es el producto punto.
- Coseno $= 1$: misma dirección, muy similares.
- Coseno $= 0$: ortogonales, no relacionados.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

BOW/TF-IDF vs Word Embeddings

Característica	**BOW**	**TF-IDF**	**Word Embeddings**
Captura orden	No	No	No (en promedio)
Captura semántica	No	No	Sí
Peso de palabras comunes	Alto	Bajo	Depende
Dimensionalidad	Alta (dispersa)	Alta (dispersa)	Baja (densa)
Interpretabilidad	Sí	Sí	Baja
Requerimiento de datos	Bajo	Bajo	Alto
Uso típico	Línea base, spam	Búsqueda, clasificación	NLP profundo

¿Cuándo usar cada una?

- **BOW**: modelos simples, línea base, interpretabilidad crucial.
- **TF-IDF**: cuando se necesita ponderar importancia, búsqueda de información, extracción de keywords.
- **Embeddings**: cuando hay suficientes datos y se requiere capturar semántica (sinónimos, analogías).

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Limitaciones

- **Maldición de la dimensionalidad**: espacio enorme, datos muy dispersos.
- **Problemas de memoria y computación**: se requieren matrices sparse.
- **Falta de generalización semántica**: sinónimos son independientes.
- **Necesidad de reducción de dimensionalidad**: LSA (SVD) para obtener representaciones densas latentes.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Aplicaciones de BOW y TF-IDF

- **Clasificación de textos**: spam, noticias, sentimiento (con Naive Bayes, SVM).
- **Búsqueda de documentos similares**: similitud del coseno para recuperación.
- **Extracción de palabras clave**: términos con mayor TF-IDF en un documento.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

CountVectorizer (BOW)

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "el gato persigue al rat n",
    "el perro persigue al gato",
    "el rat n huye del gato"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

TfidfVectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(norm='l2', use_idf=True, smooth_idf=True)
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())  # valores TF-IDF normalizados L2
```

Uso de n-gramas

```
# Bigramas (unigramas y bigramas)
vectorizer = CountVectorizer(ngram_range=(1,2))
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
```

Ejemplo con dataset real (20 Newsgroups)

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

categories = ['alt.atheism', 'soc.religion.christian']
newsgroups = fetch_20newsgroups(categories=categories)

pipeline = make_pipeline(TfidfVectorizer(), MultinomialNB())
pipeline.fit(newsgroups.data, newsgroups.target)
```

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Preguntas típicas

- Compara Bag of Words, TF-IDF y word embeddings. ¿Cuándo usarías cada uno?
- ¿Qué es IDF y por qué se usa?
- ¿Cómo calcularías la similitud entre dos documentos con TF-IDF?
- ¿Qué problemas tiene la representación BOW?
- ¿Qué son los n-gramas y para qué sirven?

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Próximos pasos

- **Sesión 4**: Clasificación de textos usando estas representaciones.
- **Sesión 5**: Word embeddings, que superan limitaciones semánticas.
- **Sesión 6**: Modelos secuenciales (RNN) que capturan orden.
- **Sesión 7**: Transformers, que usan subword tokenization y embeddings.

Contenido

1. Introducción
2. Bag of Words (BOW)
3. N-gramas
4. TF-IDF
5. Comparación con Embeddings
6. Limitaciones de Representaciones Dispersas
7. Aplicaciones Prácticas
8. Implementación en Python con scikit-learn
9. Preparación para Entrevistas
10. Conexión con el resto del curso
11. Resumen

Lo que hemos aprendido

Conceptos clave

- **Bag of Words (BOW)**: vector de frecuencias de términos. Simple, pero limitado (pérdida de orden, alta dimensionalidad, falta de semántica).
- **N-gramas**: extienden BOW capturando contexto local; aumentan dimensionalidad.
- **TF-IDF**: pondera términos según importancia en documento y rareza en corpus. Penaliza palabras comunes.
- Representaciones dispersas: sufren maldición de dimensionalidad, pero son interpretables y útiles para clasificación y búsqueda.
- Implementación con scikit-learn: `CountVectorizer`, `TfidfVectorizer`, n-gramas.

¡Gracias!